FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

4

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Student Name	Lance Kirsten Y. Buenaventura
Section:	AN41
Professor:	Ms. Crisola G. Tan
Date:	October 3, 2026

I. BACKGROUND INFORMATION

Knowledge Representation (KR) is the method of organizing facts, rules, and relationships so an AI system can understand and reason about the world.

Reasoning (R) allows the system to infer, make decisions, and solve problems based on that knowledge. Two major reasoning approaches from the module:

1. Rule-Based Reasoning (RBR)

Uses IF–THEN rules, a Knowledge Base, Working Memory, and an Inference Engine.

2. Case-Based Reasoning (CBR)

Solves new problems using past cases using the 4R Cycle:

Retrieve → Reuse → Revise → Retain

This laboratory lets you build small working implementations of both reasoning methods.

II. TECHNICAL ASSESSMENT

IMPLEMENTING KNOWLEDGE REPRESENTATION, RULE-BASED REASONING (RBR), AND CASE-BASED REASONING (CBR)

Objective

At the end of this laboratory activity, students are expected to:

1. Construct a simple knowledge base using Python data structures.
2. Implement a basic Rule-Based Reasoning (RBR) system using IF–THEN logic.
3. Build a simple Case-Based Reasoning (CBR) system using similarity matching.
4. Compare how RBR and CBR behave when solving the same type of problem.
5. Document observations and insights on KR, RBR, and CBR.

Materials Needed

1. Hardware
   - A laptop or desktop computer
   - Stable internet connection (for research, documentation, submission)
2. Software

A programming environment:

- Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)
- Any text editor (Notepad++, Sublime, VS Code) if needed

3. File Formats & Tools

Word processor for documentation:

- MS Word, Google Docs, or LibreOffice

- Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)

General Instructions

1. This **Technical Assessment (TA3)** must be completed by Pair.
2. Include clear comments in your code to describe its logic.
3. Each program should display the correct input, process, and output.
4. Ensure that your script displays clear and readable outputs.
5. Screenshot of program output for RBR
6. Screenshot of program output for CBR
7. Reflection answers
8. Short description of your knowledge base and use case
9. Compile all Python codes and corresponding outputs into one PDF file.
10. Save it as **TA4_lastname1.pdf**
    (Example: TA2_Patilano.pdf)
11. Submit the **python file** and **PDF** file.

LABORATORY TASKS

PART 1 — Knowledge Representation (KR) Setup

Task 1A — Create a Knowledge Base

Create a Python dictionary storing at least:

- 5 facts about an object (e.g., weather, appliances, students, medical symptoms)
- 3 categories (e.g., temperature, status, presence, actions)

Example structure to follow:

```
facts = {
    "temperature": 32,
    "is_raining": False,
    "humidity": 85,
}
```

PART 2 — Implement Rule-Based Reasoning (RBR)

Task 2A — Create at least 3 IF–THEN rules

Write Python functions representing rules.

Example:

```
def rule_hot(facts):
    if facts["temperature"] > 30:
        return "Turn on the aircon"
```

Task 2B — Implement an Inference Engine

Create a loop that:

1. Reads the knowledge base
2. Checks each rule

3. Returns all actions inferred

Task 2C — Display Output

Example [Program should print:]
Rule-Based Reasoning Actions: - Turn on the aircon - Bring umbrella

PART 3 — Implement Case-Based Reasoning (CBR)

Task 3A — Build a Case Base

Create at least **3 past cases**, each containing:

- A set of problem features
- A matching solution

Example:

```
case_base = [
    {"problem": {"won't turn on": True, "noise": False},
     "solution": "Check power cable."}
]
```

Task 3C — Reuse Step

Return the solution of the most similar case.

Task 3D — Revise Step

Allow student to adjust solution manually or return unchanged.

Task 3E — Retain Step

Add the new case to the case base.

PART 4 — Comparative Reflection

Write a short paragraph answering:

1. Which system (RBR or CBR) was easier to implement? Why?

    - RBR was easier for me to implement because each rule follows a clear IF–THEN condition. For example, if it is raining, the program recommends closing the windows. CBR needed more steps, including comparing cases, reviewing a previous solution, and saving the revised case.

2. Which is more flexible in solving unstructured problems?

- CBR is generally more flexible when a problem does not fit a fixed rule exactly. It can retrieve a similar situation and let me adjust its solution. However, my program still needs structured inputs, such as temperature and occupancy, to compare cases.

3. Which is better for situations with continuously changing data?

    - CBR can be more adaptable when new situations and useful solutions keep appearing because it can save them for future use. However, saved solutions need careful review since an incorrect recommendation could also be reused. RBR still works well when input values change but the rules remain valid.

4. Examples of real-world applications for both.

    - For the two of them, RBR is used in systems such as thermostats and equipment alarms, where specific conditions trigger actions. CBR is used in technical support and equipment troubleshooting, where solutions from similar past problems help resolve new ones. My study-room assistant demonstrates both approaches using everyday room conditions.

Knowledge Base and Use Case

I designed the knowledge base around a familiar situation: setting up a comfortable room for studying. It contains seven facts across four categories—weather, occupancy, lighting, and personal preferences. These include the room temperature, whether someone is inside, and my preferred temperature. The assistant uses rules to explain its recommendations and past cases to suggest solutions I can revise and save. This makes the project feel connected to my own study routine.

Your Code - Laboratory Tasks ⇩ + Screenshots

```
#Lance Kirsten Y. Buenaventura - AN41
#Technical Assessment 4


import argparse
from copy import deepcopy
import json
import math
import os
from pathlib import Path
import tempfile


def print_section(title):
```

```
    """Separate major sections so their outputs are easy to screenshot."""
    print("\n" + "=" * 72)
    print(title)
    print("=" * 72)


def print_step(title):
    print("\n" + "-" * 72)
    print(title)
    print("-" * 72)


def print_input_summary(facts):
        """Repeat compact inputs in each reasoning section for standalone
screenshots."""
    weather = facts["weather"]
    preferences = facts["preferences"]
    yes_no = lambda value: "Yes" if value else "No"
    print(f"INPUT: Temperature {weather['temperature']:g} C | "
                                f"Humidity   {weather['humidity']:g}%   |   Rain:
{yes_no(weather['is_raining'])}")
    print(f"       Occupied: {yes_no(facts['occupancy']['is_occupied'])} | "
          f"Dark: {yes_no(facts['lighting']['is_dark'])}")
                print(f"                                Preferred    temperature:
{preferences['preferred_temperature']:g} C | "
          f"Natural light: {yes_no(preferences['prefer_natural_light'])}")


# PART 1: Seven facts in four categories. Preferences are editable examples,
# not assumptions about the student's actual preferred study conditions.
FACTS = {
    "weather": {"temperature": 32.0, "humidity": 85.0, "is_raining": True},
    "occupancy": {"is_occupied": True},
    "lighting": {"is_dark": True},
       "preferences": {"preferred_temperature": 26.0, "prefer_natural_light":
True},
}

CASE_BASE = [
    {
        "name": "Hot, humid, rainy evening",
        "problem": {
            "weather": {
                "temperature": 34.0, "humidity": 90.0, "is_raining": True,
            },
            "occupancy": {"is_occupied": True},
            "lighting": {"is_dark": True},
                                "preferences":  {"preferred_temperature":  26.0,
"prefer_natural_light": True},
        },
            "solution": "Turn on the aircon, close windows, and turn on the
lights.",
    },
    {
        "name": "Comfortable, dry daytime",
        "problem": {
```

```
            "weather": {
                "temperature": 25.0, "humidity": 50.0, "is_raining": False,
            },
            "occupancy": {"is_occupied": True},
            "lighting": {"is_dark": False},
                                "preferences":  {"preferred_temperature":  26.0,
"prefer_natural_light": True},
        },
        "solution": "Keep cooling off and use natural light.",
    },
    {
        "name": "Empty room on a rainy evening",
        "problem": {
            "weather": {
                "temperature": 27.0, "humidity": 75.0, "is_raining": True,
            },
            "occupancy": {"is_occupied": False},
            "lighting": {"is_dark": True},
                                "preferences":  {"preferred_temperature":  26.0,
"prefer_natural_light": True},
        },
          "solution": "Close windows; keep lights and cooling off in the empty
room.",
    },
]


# PART 2: Each IF-THEN function returns an action when its condition is true.
def rule_hot(facts):
    current = facts["weather"]["temperature"]
    preferred = facts["preferences"]["preferred_temperature"]
    if current > preferred and facts["occupancy"]["is_occupied"]:
            return (f"Turn on the aircon toward {preferred:g} C because the
occupied "
                f"room is {current:g} C, above your preferred temperature.")


def rule_raining(facts):
    if facts["weather"]["is_raining"]:
        return "Close the windows because it is raining."


def rule_dark(facts):
    if facts["lighting"]["is_dark"] and facts["occupancy"]["is_occupied"]:
        return "Turn on the study light because the room is dark and occupied."


def rule_study_light(facts):
    if not facts["lighting"]["is_dark"] and facts["occupancy"]["is_occupied"]:
        if facts["preferences"]["prefer_natural_light"]:
            return "Use available daylight because the room is bright and you
prefer natural light."
        return "Use your study lamp because you prefer it to natural light."


def rule_empty(facts):
```

```
    if not facts["occupancy"]["is_occupied"]:
        return "Keep lights and cooling off because the room is unoccupied."


RULES = [rule_hot, rule_raining, rule_dark, rule_study_light, rule_empty]


def infer_actions(facts):
        """Check every rule against a working-memory snapshot; collect all
actions."""
    working_memory = deepcopy(facts)
    actions = []
    for rule in RULES:
        action = rule(working_memory)
        print(f"  {rule.__name__}: {'MATCH' if action else 'no match'}")
        if action:
            actions.append(action)
    return actions


# PART 3: Retrieve, Reuse, Revise, Retain.
def feature_scores(new_problem, past_problem):
        """Return named scores for seven equally weighted facts, including
preferences.

    Numeric score = max(0, 1 - absolute difference / scale).
    Scales: 40 degrees Celsius for temperature, 100 points for humidity.
    Boolean features score 1 for an exact match and 0 otherwise.
    These scales are demonstration choices, not learned parameters.
    """
    new_weather = new_problem["weather"]
    old_weather = past_problem["weather"]
    return {
                "temperature": max(0.0, 1 - abs(new_weather["temperature"] -
old_weather["temperature"]) / 40),
                    "humidity": max(0.0, 1 - abs(new_weather["humidity"] -
old_weather["humidity"]) / 100),
        "rain": float(new_weather["is_raining"] == old_weather["is_raining"]),
                 "occupancy": float(new_problem["occupancy"]["is_occupied"] ==
past_problem["occupancy"]["is_occupied"]),
                     "darkness": float(new_problem["lighting"]["is_dark"] ==
past_problem["lighting"]["is_dark"]),
        "preferred temperature": max(0.0, 1 - abs(
            new_problem["preferences"]["preferred_temperature"]
            - past_problem["preferences"]["preferred_temperature"]
        ) / 40),
        "natural light preference": float(
            new_problem["preferences"]["prefer_natural_light"]
            == past_problem["preferences"]["prefer_natural_light"]
        ),
    }


def similarity(new_problem, past_problem):
    scores = feature_scores(new_problem, past_problem)
    return sum(scores.values()) / len(scores)
```

```
def retrieve_case(problem, case_base):
      """Score all cases; ties prefer the latest case so saved revisions can
win."""
    if not case_base:
        raise ValueError("Retrieval requires at least one past case.")
    best_case, best_score = None, -1.0
    for case in case_base:
        # Older runs may have saved a menu number as the solution. Preserve
        # those records, but do not reuse them as study recommendations.
        if not is_written_solution(case["solution"]):
                print(f"  Skipping {case['name']}: {case['solution']!r} is not a
written action.")
            continue
        score = similarity(problem, case["problem"])
        print(f"  {case['name']}: {score:.1%} similarity")
        if score >= best_score:
            best_case, best_score = case, score
    if best_case is None:
          raise ValueError("No case contains a written solution. Use --demo for
the sample cases.")
    return best_case, best_score


def reuse_solution(case):
    return case["solution"]


def revise_solution(solution, replacement=""):
    """A blank manual revision keeps the retrieved solution unchanged."""
    return replacement.strip() or solution


def is_written_solution(solution):
       """Reject blank or numeric-only answers; this does not judge their
meaning."""
       return isinstance(solution, str) and any(char.isalpha() for char in
solution)


def review_solution(solution, example):
    """Separate choosing a menu option from writing a replacement action."""
    print(f"Suggested solution: {solution}")
    while True:
        print("\nWhat would you like to do?")
        print("  1 - KEEP the suggested solution (no writing needed).")
        print("  2 - EDIT it by writing your own action or sentence.")
        choice = input("Choose 1 or 2, or press Enter to keep: ").strip()
        if choice in ("", "1"):
            print("Revision: kept the suggested solution unchanged.")
            return revise_solution(solution)
        if choice == "2":
                print("\nWrite the action you recommend for the CURRENT room
conditions.")
            print(f"Example: {example}")
```

```
            print("Write words here, not an option number. Enter alone returns
to the menu.")
        while True:
            replacement = input("My revised solution: ").strip()
            if not replacement:
                break
            if is_written_solution(replacement):
                print("Revision: replaced the suggestion with your written
solution.")
                return revise_solution(solution, replacement)
            print("Please write an action, such as 'Close the windows.' A
number is not a solution.")
    else:
        print("Please enter 1 to keep or 2 to edit. You can write your
solution AFTER choosing 2.")


def retain_case(problem, solution, case_base):
    # Copy the facts so later changes cannot alter the retained problem.
    case_base.append({
        "name": f"Retained case {len(case_base) + 1}",
        "problem": deepcopy(problem),
        "solution": solution,
    })


def validate_cases(cases):
    """Reject malformed files before reasoning or overwriting stored data."""
    if not isinstance(cases, list) or len(cases) < 3:
        raise ValueError("The saved case base must contain at least three
cases.")
    for case in cases:
        if not isinstance(case, dict):
            raise ValueError("Each saved case must be an object.")
        for field in ("name", "solution"):
            if not isinstance(case.get(field), str) or not case[field].strip():
                raise ValueError(f"Each case requires a nonempty {field}.")
        problem = case.get("problem")
        if not isinstance(problem, dict):
            raise ValueError("Each case requires problem features.")
        for category, features in FACTS.items():
            values = problem.get(category)
            if not isinstance(values, dict):
                raise ValueError(f"Missing category: {category}.")
            for key, example in features.items():
                value = values.get(key)
                if isinstance(example, bool):
                    valid = isinstance(value, bool)
                else:
                    low, high = (0, 100) if key == "humidity" else (-20, 60)
                    valid = (type(value) in (int, float)
                             and low <= value <= high)
                if not valid:
                    raise ValueError(f"Invalid saved feature:
{category}.{key}.")
```

```
def load_cases(path):
    if not path.exists():
        return deepcopy(CASE_BASE)
    cases = json.loads(path.read_text(encoding="utf-8"))
    validate_cases(cases)
    return cases


def save_cases(path, cases):
    """Write a complete temporary file before replacing the saved case base."""
    validate_cases(cases)
    temporary = None
    try:
        with tempfile.NamedTemporaryFile(
            mode="w", encoding="utf-8", dir=path.parent,
            prefix=path.name + ".", suffix=".tmp", delete=False,
        ) as handle:
            temporary = Path(handle.name)
            json.dump(cases, handle, indent=2, allow_nan=False)
            handle.write("\n")
        os.replace(temporary, path)
    finally:
        if temporary is not None and temporary.exists():
            temporary.unlink()


def read_number(prompt, minimum, maximum):
    while True:
        try:
            value = float(input(prompt))
            if math.isfinite(value) and minimum <= value <= maximum:
                return value
        except ValueError:
            pass
        print(f"Enter a number from {minimum} to {maximum}.")


def read_bool(prompt):
    while True:
        answer = input(prompt + " [y/n]: ").strip().lower()
        if answer in ("y", "yes", "n", "no"):
            return answer in ("y", "yes")
        print("Enter y or n.")


def read_facts():
    print("For temperatures and humidity, enter numbers only (example: 25).")
    print("For yes/no questions, type y for yes or n for no.\n")
    return {
        "weather": {
             "temperature": read_number("Temperature in Celsius (-20 to 60): ",
-20, 60),
            "humidity": read_number("Relative humidity (0 to 100): ", 0, 100),
            "is_raining": read_bool("Is it raining?"),
        },
```

```
        "occupancy": {"is_occupied": read_bool("Is the room occupied?")},
        "lighting": {"is_dark": read_bool("Is the room dark?")},
        "preferences": {
                "preferred_temperature": read_number("Preferred temperature in
Celsius (-20 to 60): ", -20, 60),
            "prefer_natural_light": read_bool("Do you prefer natural light when
available?"),
        },
    }


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    modes = parser.add_mutually_exclusive_group()
      modes.add_argument("--interactive", action="store_true", help="Enter your
own facts (the default).")
       modes.add_argument("--demo", action="store_true", help="Run the example
without prompts.")
    parser.add_argument(
        "--case-file", type=Path,
        default=Path(__file__).resolve().with_name("study_cases.json"),
        help="Saved case file (default: study_cases.json beside this script).",
    )
    args = parser.parse_args()

    print_section("TA4 - LANCE'S STUDY-ROOM ASSISTANT")
    print("Use case: suggest room settings for a comfortable study session.")
    try:
                        case_base = deepcopy(CASE_BASE) if args.demo else
load_cases(args.case_file)
    except (OSError, ValueError) as error:
        parser.exit(1, f"Cannot load saved cases: {error}\n"
                           "File left unchanged. Use --demo or --case-file with
another path.\n")
    print_section("ENTER YOUR ROOM CONDITIONS AND STUDY PREFERENCES")
    if args.demo:
                print("Using example preferences: 26 C and natural light when
available.")
                  print("Run normally to enter your own room conditions and
preferences.")
    facts = deepcopy(FACTS) if args.demo else read_facts()
    print_section("PART 1 - KNOWLEDGE BASE / INPUT (7 facts, 4 categories)")
    for category, features in facts.items():
        print(f"  {category.title()}:")
        for name, value in features.items():
            print(f"    {name}: {value}")

    print_section("PART 2 - RULE-BASED REASONING (RBR)")
    print_input_summary(facts)
    print_step("PROCESS - Check each IF-THEN rule")
    actions = infer_actions(facts)
    print_step("OUTPUT - Rule-Based Reasoning Actions")
    for action in actions or ["No rule matched; no action inferred."]:
        print(f"- {action}")
    print("-" * 72)
    print("END OF RBR OUTPUT")
```

```
    print_section("PART 3 - CASE-BASED REASONING (CBR)")
    print_input_summary(facts)
    print_step("1. RETRIEVE - Compare all seven features against past cases")
    try:
        case, score = retrieve_case(facts, case_base)
    except ValueError as error:
        parser.exit(1, f"Cannot retrieve a case: {error}\n")
    print(f"Selected: {case['name']} ({score:.1%} similarity)")
      print("Why this case: highest average feature score; latest case wins
ties.")
    for feature, match in feature_scores(facts, case["problem"]).items():
        print(f"  {feature}: {match:.1%} match")
    solution = reuse_solution(case)
    print_step("2. REUSE - Start with the selected case's solution")
    print(solution)
    print_step("3. REVISE - Keep or edit the suggested solution")
    if args.demo:
        revised = revise_solution(solution)
        print("Demo: keeping the suggested solution unchanged.")
    else:
        example = " ".join(actions) or "Keep the current room settings."
        revised = review_solution(solution, example)
    print_step("OUTPUT - Final CBR solution")
    print(revised)
    print_step("4. RETAIN - Add the current problem and final solution")
    before = len(case_base)
    retain_case(facts, revised, case_base)
    print(f"Case base updated: {before} -> {len(case_base)} cases.")
    if args.demo:
        print("Demo only: retained in memory; saved cases are unchanged.")
    else:
        try:
            save_cases(args.case_file, case_base)
        except (OSError, ValueError) as error:
                parser.exit(1, f"Case retained in memory but could not be saved:
{error}\n")
        print(f"Saved to {args.case_file.resolve()} for future runs.")
    print("-" * 72)
    print("END OF CBR OUTPUT")

    print_section("COMPARISON - RBR AND CBR")
    print(f"RBR inferred {len(actions)} action(s) from explicit conditions.")
       print("CBR reused one similar case; manual revision can adapt its
solution.")
        print("Similarity is a feature-match score, not a guarantee of
correctness.")
    print("=" * 72)


if __name__ == "__main__":
    try:
        main()
    except (EOFError, KeyboardInterrupt):
        print("\nInput cancelled. Run with --demo for the built-in example.")
```

Screenshots:

PART 1 - User Input

```
======================================================================
TA4 - LANCE'S STUDY-ROOM ASSISTANT
======================================================================
Use case: suggest room settings for a comfortable study session.

======================================================================
ENTER YOUR ROOM CONDITIONS AND STUDY PREFERENCES
======================================================================
For temperatures and humidity, enter numbers only (example: 25).
For yes/no questions, type y for yes or n for no.

Temperature in Celsius (-20 to 60): 22
Relative humidity (0 to 100): 40
Is it raining? [y/n]: y
Is the room occupied? [y/n]: n
Is the room dark? [y/n]: n
Preferred temperature in Celsius (-20 to 60): 24
Do you prefer natural light when available? [y/n]: y

======================================================================
PART 1 - KNOWLEDGE BASE / INPUT (7 facts, 4 categories)
======================================================================
  Weather:
    temperature: 22.0
    humidity: 40.0
    is_raining: True
```

```
======================================================================
PART 1 - KNOWLEDGE BASE / INPUT (7 facts, 4 categories)
======================================================================
  Weather:
    temperature: 22.0
    humidity: 40.0
    is_raining: True
  Occupancy:
    is_occupied: False
  Lighting:
    is_dark: False
  Preferences:
    preferred_temperature: 24.0
    prefer_natural_light: True
```

PART 2 - RBR Screenshot

```
======================================================================
PART 2 - RULE-BASED REASONING (RBR)
======================================================================
INPUT: Temperature 22 C | Humidity 40% | Rain: Yes
       Occupied: No | Dark: No
       Preferred temperature: 24 C | Natural light: Yes

----------------------------------------------------------------------
PROCESS - Check each IF-THEN rule
----------------------------------------------------------------------
  rule_hot: no match
  rule_raining: MATCH
  rule_dark: no match
  rule_study_light: no match
  rule_empty: MATCH

----------------------------------------------------------------------
OUTPUT - Rule-Based Reasoning Actions
----------------------------------------------------------------------
- Close the windows because it is raining.
- Keep lights and cooling off because the room is unoccupied.
----------------------------------------------------------------------
END OF RBR OUTPUT
```

PART 3 - CBR Screenshots

```
======================================================================
PART 3 - CASE-BASED REASONING (CBR)
======================================================================
INPUT: Temperature 22 C | Humidity 40% | Rain: Yes
       Occupied: No | Dark: No
       Preferred temperature: 24 C | Natural light: Yes

----------------------------------------------------------------------
1. RETRIEVE - Compare all seven features against past cases
----------------------------------------------------------------------
  Hot, humid, rainy evening: 59.3% similarity
  Comfortable, dry daytime: 68.2% similarity
  Empty room on a rainy evening: 78.2% similarity
Selected: Empty room on a rainy evening (78.2% similarity)
Why this case: highest average feature score; latest case wins ties.
  temperature: 87.5% match
  humidity: 65.0% match
  rain: 100.0% match
  occupancy: 100.0% match
  darkness: 0.0% match
  preferred temperature: 95.0% match
  natural light preference: 100.0% match

----------------------------------------------------------------------
2. REUSE - Start with the selected case's solution
----------------------------------------------------------------------
Close windows; keep lights and cooling off in the empty room.
```

```
----------------------------------------------------------------------
3. REVISE - Keep or edit the suggested solution
----------------------------------------------------------------------
Suggested solution: Close windows; keep lights and cooling off in the empty room.

What would you like to do?
  1 - KEEP the suggested solution (no writing needed).
  2 - EDIT it by writing your own action or sentence.
Choose 1 or 2, or press Enter to keep: 2

Write the action you recommend for the CURRENT room conditions.
Example: Close the windows because it is raining. Keep lights and cooling off because the room is unoccupied.
Write words here, not an option number. Enter alone returns to the menu.
My revised solution: Close windows but keep cooling on to keep the room cool.
Revision: replaced the suggestion with your written solution.

----------------------------------------------------------------------
OUTPUT - Final CBR solution
----------------------------------------------------------------------
Close windows but keep cooling on to keep the room cool.

----------------------------------------------------------------------
4. RETAIN - Add the current problem and final solution
----------------------------------------------------------------------
Case base updated: 3 -> 4 cases.
Saved to C:\Users\Lance Buenaventura\Desktop\Intelligent Systems Projects\TA4-Buenaventura\study_cases.json for future runs.
----------------------------------------------------------------------
```

```
--------------------------------------------------------------------------------
END OF CBR OUTPUT

================================================================================
COMPARISON - RBR AND CBR
================================================================================
RBR inferred 2 action(s) from explicit conditions.
CBR reused one similar case; manual revision can adapt its solution.
Similarity is a feature-match score, not a guarantee of correctness.
================================================================================
PS C:\Users\Lance Buenaventura\Desktop\Intelligent Systems Projects\TA4-Buenaventura>
```

Saved Study Cases (study_cases.json)

